Identifying, Exploring, and Understanding the DRAM Communication Scheduling Space for DNN Accelerators

ABSTRACT

Modern Deep Neural Network (DNN) accelerators are equipped with increasingly larger on-chip buffers to provide more opportunities to optimize the increasingly severe DRAM bandwidth pressure. However, current research utilizing buffer optimizations for DRAM communication still primarily focuses on single-layer dataflow scheduling optimization. As buffers grow large enough to accommodate most single-layer weights in most networks, the impact of single-layer dataflow optimization on DRAM communication diminishes significantly. Therefore, developing new paradigms to fully leverage the increasingly abundant on-chip buffer resources to optimize the increasingly constrained DRAM bandwidth has become particularly important, which remains an open challenge.

To solve this challenge, we first identify the optimization opportunities in DRAM communication scheduling by analyzing the shortcomings of existing works on the layer fusion optimization paradigm and recognizing the vast potential of the prefetching-and-delayed-storing optimization paradigm based on two key observations. To fully exploit these optimization opportunities, we first develop an encoding method and a corresponding parsing method to represent different DRAM communication scheduling schemes and depict the overall space of DRAM communication scheduling. Then, to thoroughly explore the space of DRAM communication scheduling for diverse accelerators and workloads, we develop an end-to-end scheduling framework, Tensor Shuttle, which will be deployed on a commercial accelerator. Compared with the state-of-the-art (SOTA) Cocco framework, Tensor Shuttle can, on average, achieve a ***× performance improvement and a ***% reduction in energy cost simultaneously. Then we leverage Tensor Shuttle to perform design space exploration and analyze the DRAM communication scheduling space, achieving some interesting insights. Moreover, Tensor Shuttle will be open-sourced.

1. INTRODUCTION

In order to process a variety of tasks with better performance and accuracy, Deep Neural Networks (DNNs) are rapidly becoming more complex and heavy [7, 8, 11, 28, 33]. Accelerators with more computing units, larger buffers, and higher memory bandwidth have been developed to accelerate these DNN workloads [14, 16, 17, 21, 24, 27].

However, under modern semiconductor processes, the rate of increase in DRAM bandwidth lags behind the rate of transistor density growth, a longstanding issue [30]. This disparity is even more pronounced in DNN accelerators [30, 39], which are more specialized and have a larger proportion of dedicated compute elements than traditional chips like CPU. As a result, DRAM communication is increasingly becoming a performance bottleneck in DNN computation [30, 39]. To mitigate this bottleneck, accelerators are equipped with increasingly larger on-chip buffers [14, 16, 17, 21, 24, 27], providing opportunities to optimize DRAM communication by exploiting reuse opportunities in DNNs. However, how to fully leveraging these buffer resources to improve the performance and energy efficiency of DNN accelerators is still an open challenge.

Several works leverage buffer resources to reduce DRAM accesses under the paradigm of "layer fusion" [1, 20, 23, 26, 32, 39]. This approach involves buffering the feature maps (fmaps) produced by earlier layers on-chip, allowing subsequent consuming layers to directly read them. This strategy avoids the overhead of first writing back to DRAM and then reading from it, thereby reducing DRAM access costs. This optimization paradigm holds immense potential; for instance, Cocco achieved performance improvements ranging from 1.89% to 50.33% by merely exploring the option of which layers to fuse. Beyond this, there are numerous dimensions worth exploring under this paradigm, such as execution order and execution granularity. However, like Cocco, most existing studies [1, 26, 32] have only focused on a small portion of this optimization space. Therefore, we believe that the complex optimization dimensions within the layer fusion paradigm have not been clearly delineated or defined, much less thoroughly explored and understood in the context of the entire layer-fusion optimization space.

While reducing DRAM access is an important approach for optimizing DRAM communication, we have identified another previously overlooked approach—prefetching and delayed storing. We focus on this approach and believe it has potential based on two insightful observations: (1) With fine-grained layer fusion, much data can be reused on-chip. Traditional double buffer prefetching and data store strategies [3, 26, 32] (i.e., prefetching the data required for the next workload and storing the current workload's output feature maps during the next workload) cannot fully utilize bandwidth, resulting in significant DRAM bandwidth idle time during intermediate tile computations (Fig. 2). (2) In modern DNN networks, the ratio of DRAM bandwidth demand to computation demand varies significantly across different layers. After layer fusion, the overall ratio of DRAM bandwidth demand to computation demand for different fused layer groups also varies greatly (as shown in Fig. 3). These

识别、探索与理解 DNN 加速器中的DRAM通信调度空间

摘要

现代深度神经网络（DNN）加速器配备了容量日益增大的片上缓冲器，这为缓解DRAM带宽压力提供了更多优化空间。然而当前利用缓冲器优化DRAM通信的研究仍主要聚焦于单层数据流调度优化。随着缓冲器容量已足以容纳大多数网络中的单层权重参数，单层数据流优化对DRAM通信的影响显著减弱。因此，开发新型范式以充分利用日益丰富的片上缓冲资源来优化日益受限的DRAM带宽，已成为亟待解决的关键挑战。

为解决这一挑战，我们首先通过分析现有层融合优化范式的研究不足，并基于两项关键观察认识到预取与延迟存储优化范式的巨大潜力，从而识别DRAM通信调度中的优化机会。为充分挖掘这些优化潜力，我们首先开发了编码方法及对应解析方法，用于表征不同DRAM通信调度方案并描绘其整体空间结构。随后，为深入探索适用于各类加速器与工作负载的DRAM通信调度空间，我们开发了端到端调度框架Tensor Shuttle（该框架将部署于商用加速器上）。与当前最先进的 SOTA Cocco框架相比，Tensor Shuttle平均可实现***×性能提升与***%能耗降低的双重优势。我们进一步利用Tensor Shuttle开展设计空间探索并分析DRAM通信调度空间，获得若干重要发现。此外，Tensor Shuttle将作为开源项目公开发布。

1. 引言

为了以更好的性能和准确性处理各种任务，深度神经网络（DNNs）正迅速变得更为复杂和庞大[7、8、11、28、33]。为加速这些 DNN 工作负载，已开发出具有更多计算单元、更大缓冲区和更高内存带宽的加速器[14、16、17、21、24、27]。

然而，在现代半导体工艺中，DRAM带宽的增长速率落后于晶体管密度的增长速率，这是一个长期存在的问题[30]。这种差异在 DNN 加速器中，这一特性表现得更为显著[30，39]，这类设备比传统CPU等芯片更具专业性，且专用计算单元占比更高。因此，DRAM通信正日益成为DNN 计算中的性能瓶颈[30，39]。为缓解这一瓶颈，加速器配备了日益增大的片上缓冲器[14，16，17，21，24，27]，这为通过利用深度神经网络中的复用机会来优化DRAM通信提供了可能。然而，如何充分利用这些缓冲资源以提升 DNN 加速器的性能与能效，仍是亟待解决的开放性挑战。

多项研究利用缓冲资源在“层融合”范式下降低DRAM访问量[1，20，23，26，32，39]。该方法通过在芯片上对早期层生成的特征图（fmaps）进行缓冲，使后续消费层可直接读取。此策略避免了先写回DRAM再读取的开销，从而降低DRAM访问成本。该优化范式潜力巨大：例如Cocco仅通过探索融合层的选择方案，就实现了1.89%至50.33%的性能提升。此外，该范式下还有诸多值得探索的维度，如执行顺序与执行粒度。然而与Cocco类似，现有研究[1，26，32]仅聚焦于该优化空间的局部区域。因此，我们认为在层融合范式中，复杂的优化维度尚未得到清晰界定或定义，更遑论在整个层融合优化空间背景下得到深入探索与理解。

虽然减少DRAM访问是优化DRAM通信的重要方法，但我们发现另一种先前被忽视的策略——预取与延迟存储。我们聚焦于该策略，并基于两项深刻观察认为其具有潜力：(1)通过细粒度层融合技术，大量数据可在芯片上实现复用。传统双缓冲预取与数据存储策略[3、26、32]（即预取下一工作负载所需数据，并在下一工作负载期间存储当前工作负载的输出特征图）无法充分利用带宽，导致中间瓦片计算阶段出现显著DRAM带宽空闲时间（图2）。(2)在现代 DNN 网络中，不同层间DRAM带宽需求与计算需求的比例存在显著差异。层融合后，不同融合层组的DRAM带宽需求与计算需求总体比例也呈现大幅波动（如图3所示）。这些

two observations indicate that with the application of layer fusion technology, DRAM bandwidth usage during the entire computation process becomes very uneven—sometimes leading to congestion due to high demand and sometimes causing bandwidth resource waste due to low demand. This motivates us to apply prefetching and delayed storing techniques to alleviate the uneven DRAM communication load. However, choosing the appropriate timing for prefetching and storing is a non-trivial problem. Clearly defining, thoroughly exploring, and understanding this issue is a significant challenge.

"Layer fusion" and "prefetching and delayed storing" each have their own optimization spaces, but they are not independent and are intricately coupled. This is reflected in the following points: 1) Both paradigms trade buffer usage for DRAM communication optimization, and 2) Layer fusion affects the types and quantities of data that need to communicate with DRAM. Therefore, we define the complex space formed by these two paradigms as the DRAM Communication Scheduling Space. From the previous introduction, it is evident that the individual spaces have not been clearly defined, let alone the definition, exploration, and understanding of this overall space.

After identifying the challenges and optimization potential of DRAM communication scheduling, we make the following contributions to address these challenges, exploit this potential, and enhance our understanding of DRAM communication scheduling.

We first introduce a comprehensive encoding method with two categories and a total of six attributes to encode the scheduling schemes in the DRAM Communication Scheduling Space, then we show how to parse each encode into actual hardware behaviours. Based on this encoding method, we define and illustrate the DRAM communication optimization space, and the complex trade-offs behind it. Existing works can all be described using our encoding method, representing merely a small subset within the space we have defined. To the best of our knowledge, this work is ***the first*** to comprehensively define and analyze the DRAM communication optimization space.

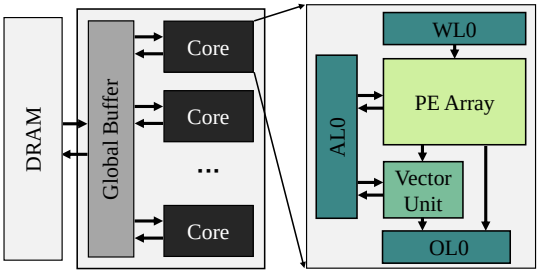
To explore the DRAM Communication Scheduling Space defined by the aforementioned encoding method, we develop an end-to-end framework, Tensor Shuttle, which is equipped with a two-stage exploration algorithm. We have successfully built a complete flow based on Tensor Shuttle, from model input to instruction generation, on a chip that is about to go into mass production.

Then, we conduct extensive experiments on different workloads, platforms, and batch sizes, demonstrating *** performance improvements compared with the state-of-the-art framework, Cocco. In addition, we use Tensor Shuttle to explore and analyze the architectural design space of the DRAM Communication Scheduling Space, gaining some interesting insights. For example, ***

2. HARDWARE BASIS

In this section, we introduce a generic large-scale DNN accelerator template adopted in this paper 1, which represents a significant portion of mainstream commercial DNN accelerators [2, 10, 15, 16, 17, 24, 27].

It primarily consists of DRAM, a Global Buffer, and several cores. The Global Buffer is shared among all cores. Each core has a private small buffer/register files for rapid memory access by internal computing units; the PE array is used for computing GEMM/Conv operations, and the Vector Unit is designed for computing other vector/scalar operations, such as element-wise add, max pooling, etc.



(a) DNN Accelerator (b) Computing Core

Figure 1: DNN Accelerator Template

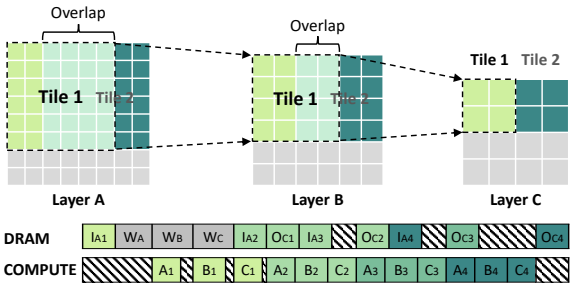


Figure 2: Layer Fusion Paradigm

3. MOTIVATION ANALYSIS

The Global Buffer plays a crucial role in optimizing DRAM communication within DNN accelerators, and research on this topic has been a hot topic since the advent of DNN accelerators. However, most current studies focus on optimizing dataflow for small-scale DNN accelerators, specifically on how to tile a single layer into small parts and adjust the computation order of these parts to optimize DRAM communication [4, 5, 9, 12, 13, 17, 18, 22, 25, 29, 31, 34, 36, 37].

As DNN accelerators have evolved, architects have equipped them with increasingly larger Global Buffers, with capacities reaching tens or even hundreds of megabytes [2, 6, 10, 15, 16, 21, 24]. Such large buffers can accommodate most layers of most networks, significantly reducing the effects of optimizing dataflow for DRAM communication. Therefore, developing new techniques to fully leverage the rapidly growing on-chip buffer resources to optimize the increasingly bottlenecked DRAM bandwidth is crucial and remains an open challenge. There are significantly fewer related articles compared to those focused on dataflow optimization for small-scale accelerators.

3.1 Layer Fusion

Layer fusion is an important paradigm for using buffers to optimize DRAM communication. Fig. 2 illustrates the fusion and computation method, where multiple fused layers are

两项观测结果表明，应用层融合技术后，DRAM带宽在整个计算过程中的使用情况呈现显著不均衡性——有时因高需求导致通信拥堵，有时又因低需求造成带宽资源浪费。这促使我们采用预取与延迟存储技术来缓解DRAM通信负载的不均衡性。然而，如何选择合适的预取与存储时机仍是一个复杂难题。明确界定、深入探索并理解这一问题具有重大挑战性。“层融合”与“预取延迟存储”各自拥有优化空间，但二者并非独立存在而是紧密耦合。具体体现在以下两点：1）两种技术范式均以缓冲区使用为代价来优化DRAM通信；2）层融合会影响需要与DRAM通信的数据类型及传输量。因此，我们将这两种技术范式构成的复杂空间定义为DRAM通信调度空间。从前述引言中可明显看出，各独立空间尚未得到明确定义，更遑论对该整体空间的定义、探索与理解。

在明确DRAM通信调度面临的挑战及优化潜力后，我们通过以下研究贡献来应对这些挑战、挖掘潜在价值，并深化对DRAM通信调度机制的理解。

我们首先提出了一种包含两类共六个属性的综合编码方法，用于对DRAM通信调度空间中的调度方案进行编码，随后详细阐述了如何将每个编码解析为实际硬件行为。基于该编码方法，我们定义并阐释了DRAM通信优化空间及其背后的复杂权衡关系。现有研究均可通过我们的编码方法进行描述，仅构成所定义空间中的一个小部分子集。据我们所知，本研究是***首次***对DRAM通信优化空间进行全面定义与分析的成果。

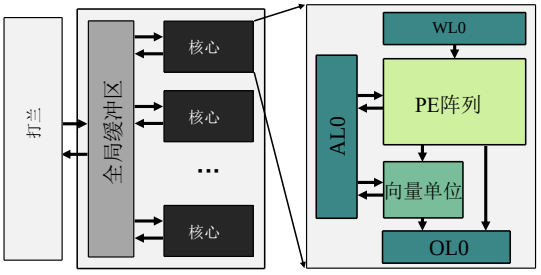
为探索上述编码方法所定义的DRAM通信调度空间，我们开发了端到端框架Tensor Shuttle，并配备两级探索算法。基于Tensor Shuttle，我们成功构建了从模型输入到指令生成的完整流程，并在即将投入大规模生产的芯片上实现了该流程。

随后，我们在不同工作负载、平台及批量处理规模下开展了大量实验，结果表明其性能较当前最先进框架Cocco有显著提升。此外，我们运用Tensor Shuttle工具对DRAM通信调度空间的架构设计空间进行探索分析，获得了若干重要发现。例如，***

2. 硬件基础

在本节中，我们介绍了一种通用的大规模 DNN 加速器模板，该模板在本文中采用1，它代表了主流商业 DNN 加速器的重要组成部分[2、10、15、16、17、24、27]。

其主要由动态随机存取存储器（DRAM）、全局缓冲区及sev-组成



(a) DNN 加速器　　(b) 计算核心

图1： DNN 加速器模板

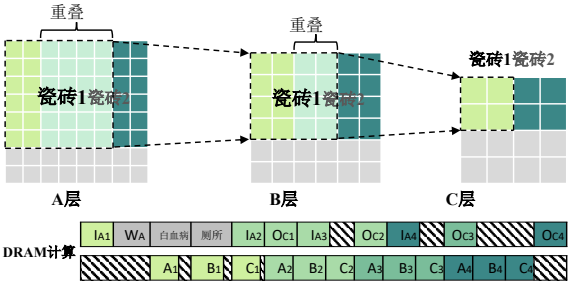


图2：层融合范式

核心核心。全局缓冲区由所有核心共享。每个核心拥有私有小型缓冲区/寄存器文件，供内部计算单元快速访问内存；处理单元阵列用于执行 GEMM /卷积运算，向量单元则专为执行其他向量/标量运算设计，例如逐元素加法、最大池化等。

3. 动机分析

全局缓冲区在优化 DNN 加速器中的DRAM通信方面起着关键作用，自 DNN 加速器问世以来，相关研究一直是热门课题。然而，当前大多数研究聚焦于小规模 DNN 加速器的数据流优化，具体而言是如何将单层划分为小部分，并调整这些部分的计算顺序以优化DRAM通信[4，5，9，12，13，17，18，22，25，29，31，34，36，37]。

随着 DNN 加速器的演进，架构师为其配备了容量日益增大的全局缓冲区，其容量可达数十甚至数百兆字节[2、6、10、15、16、21、24]。如此庞大的缓冲区能够容纳大多数网络的各层结构，显著降低了针对DRAM通信进行数据流优化的影响。因此，开发新技术以充分利用快速增长的片上缓冲资源来优化日益受限的DRAM带宽至关重要，这仍是一个亟待解决的挑战。与专注于小规模加速器数据流优化的研究相比，相关文献数量明显较少。

3.1 层融合

层融合是利用缓冲区优化DRAM通信的重要范式。图2展示了融合与计算方法，其中包含多个融合层。

partially computed in series, relying on on-chip buffers for switching fmaps. The optimization dimensions within this paradigm are numerous and complex, creating a vast optimization space. A straightforward optimization dimension is which layers to fuse, which affects both DRAM access savings and buffer occupancy. This is the primary focus of most existing studies [32, 38]. Another evident optimization space is the computation granularity of each fused layer group, i.e., the size of each part (Fig. **??**). This affects buffer occupancy, halo overlap overhead, and the effectiveness of intra-core optimizations (detailed analysis in Sec. **??**). DeFENIS [26] has analyzed this dimension. Most other studies address this issue using heuristic rules [32, 38], overlooking optimization opportunities within this space. These two dimensions are just simple examples; there are other optimization dimensions neglected by existing studies (introduced in Sec. 4.1.1). Even for these two dimensions, most existing work focuses on one while adopting heuristic strategies for the other [26, 32, 38].

figure 2-3 unblance demand

3.2 Prefetching and Delayed Storing

Besides layer fusion, we have identified another optimization opportunity overlooked by existing literature—prefetching and delayed storing. We believe this optimization opportunity also has great potential based on the following two insightful observations.

The first observation is that, in terms of total volume, the ratio of DRAM access demand to compute demand varies greatly across different fused layer groups (as shown by the red and blue dots in Fig. **??**). This results in significant differences in DRAM bandwidth requirements for different fused layer groups. The fundamental factor leading to this phenomenon is that in modern DNNs, the DRAM access demand and compute demand vary greatly among different layers. As shown in Fig. 2, the layer with the highest DRAM access/computation time ratio in Transformer and ResNet-50 is 2.7 times and 5 times that of the layer with the lowest ratio, respectively.

The second observation is that, from a micro perspective, the existing prefetching and storing strategy (double buffer strategy) leads to uneven DRAM bandwidth usage within a fused layer group in the layer fusion scenario. Specifically, as shown in Fig. 2, the first tile of most layers might experience congestion due to prefetching weights, while subsequent tiles are likely to be idle because the weights are already on-chip and many feature maps can be reused on-chip.

Combining these two observations, we find that controlling the timing of data prefetching and storing to utilize idle DRAM bandwidth and alleviate peak-time pressure has great potential. However, achieving precise control over this process is non-trivial. Clearly defining, thoroughly exploring, and understanding this issue is a significant challenge.

3.3 Combine Them Together

"Layer fusion" and "prefetching and delayed storing" are two optimization paradigms with complex interrelationships, which lies in two aspects. The first aspect lies in the fact that both optimization paradigms inherently trade buffer usage for DRAM communication optimization, leading to competition for buffer resources between the two. Additionally, layer fusion affects the tensors that need DRAM communication, which in turn impacts the optimization space for prefetching and delayed storing. As introduced before, these interdependent optimization spaces have yet to be clearly defined individually, let alone the overall optimization space they constitute. This motivates us to clearly define the overall optimization space, thoroughly explore it, and thereby enhance our understanding of this space.

4. DELINEATION AND ANALYSIS OF DRAM COMMUNICATION OPTIMIZATION SPACE

4.1 Encoding Format and Parsing Methods

In this section, we will introduce the semantics of our encoding method, how it translates into actual hardware behaviors, and the trade-offs behind different encoding choices through a practical example of a five-layer network (A to E) shown in Fig. 3.

As shown in Fig. 3, the encoding format has six attributes, which can be divided into two categories: Layer-Fusion-related Attributes (LFA) and DRAM-Load-and-store-related Attributes (DLSA). Consequently, the overall parsing process is also divided into two stages. The first stage involves parsing the LFA to determine: 1) the computation granularity and order, as well as the buffer occupancy of tensors that are reused on-chip, and 2) all tiles that need to interact with DRAM. The second stage involves parsing the DLSA to derive the specific timing and buffer occupancy details for all tensors that need to be loaded/stored to/from DRAM.

4.1.1 Layer-Fusion-related Attributes

LFA includes ***Layer Order, Fine-grained Layer-fusion Cut (FLC), Tile Number, and DRAM Cut (DC)***, which will be introduced one by one.

The first attribute, Layer Order, arranges these layers into a serial sequence, representing their coarse-grained execution order. Hence, a valid layer order cannot have any dependency that goes from right to left, as this would result in a scenario where the data needed by a layer calculated earlier is not yet computed. Its format is a vector In Fig. 3 is an example of an order; swapping the order of D and E would still result in a valid order, but swapping A and B would not be valid.

The second attribute, Fine-grained Layer-fusion Cut (FLC), records the cut locations, which divides layers into Fine-grained Layer-fusion Groups (FLG). All FLCs constitute the FLC set (FLCS). Each FLG possesses a **Tile Number attribute**, which determines the computing granularity of the FLG. Layers within an FLG are processed continuously at the granularity of tiles rather than completely finishing the first layer before starting the second, thereby saving on-chip buffer overhead. For the example in the left top of Fig. 3, the layers are cut into three FLGs (A,B,C,E,D) with Tile Number $1,2,2$, respectively. Within the FLG (C, D, E), each layer is segmented into two tiles which are then processed in sequence. We use a heuristic strategy to allocate the tile number's division dimensions to each layer. This strategy prioritizes splitting the batch, then the height and width of the output feature maps (keeping them as equal as possible to reduce overlap). The reason for not splitting the channel dimension is that splitting the channel would prevent the next

部分计算采用串行方式，依赖片上缓冲器进行功能映射切换。该范式中的优化维度众多且复杂，形成了庞大的优化空间。一个直观的优化维度是选择哪些层进行融合，这既影响DRAM访问节省率又关系到缓冲器占用率。这正是大多数现有研究[32，38]的主要关注点。另一个显著的优化空间是每个融合层组的计算粒度，即各部分的尺寸（图？？）。这会影响缓冲器占用率、晕轮重叠开销以及核心内优化效果（详见第？？节）。DeFENIS[26]已对此维度进行过分析。多数其他研究采用启发式规则处理该问题[32，38]，却忽视了该维度中的优化机会。这两个维度仅是简单示例；现有研究还忽略了其他优化维度（详见第4.1.1节）。即使针对这两个维度，现有研究大多只关注其中一个维度，同时对另一个维度采用启发式策略[26，32，38]。

图2-3 不均衡需求

3.2 预取与延迟存储

除层融合外，我们还发现了一个被现有文献忽视的优化机会——预取与延迟存储。基于以下两项深刻观察，我们认为该优化机会同样具有巨大潜力。

第一个观察结果是：从总体积来看，不同融合层组中DRAM访问需求与计算需求的比例存在显著差异（如图？？中的红蓝圆点所示）。这导致不同融合层组对DRAM带宽的需求存在明显差别。造成这一现象的根本原因在于现代深度神经网络中，各层间的DRAM访问需求与计算需求存在巨大差异。如图2所示，在Transformer和ResNet-50模型中，DRAM访问/计算时间比最高的层分别是最低比值层的2.7倍和5倍。

第二个观察结果是：从微观层面来看，在层融合场景中，现有的预取与存储策略（双缓冲策略）会导致融合层组内DRAM带宽使用不均衡。具体而言，如图2所示，由于预取权重的存在，多数层的首层瓦片可能出现带宽拥堵，而后续瓦片则可能处于空闲状态——因为权重数据已存储在芯片上，且大量特征图可实现芯片级复用。

综合两项研究发现，通过精准调控数据预取与存储的时机，可有效利用空闲DRAM带宽并缓解峰值时段压力，这一策略具有显著应用潜力。然而，实现该过程的精确控制并非易事。明确界定、深入探究并全面理解该技术问题，仍是当前亟待攻克的重大挑战。

3.3 将二者结合

“层融合”与“预取与延迟存储”是两种具有复杂相互关系的优化范式，其差异主要体现在两个层面。首先，这两种优化范式本质上都以缓冲区使用量为代价来优化DRAM通信性能，导致两者对缓冲资源产生竞争。此外，层数据融合会对需要DRAM通信的张量产生影响，进而影响预取与延迟存储的优化空间。如前所述，这些相互依存的优化空间尚未被单独明确界定，更不用说它们共同构成的整体优化空间了。这促使我们有必要对整体优化空间进行清晰界定、深入探索，从而深化对该领域的认知。

4. 戏剧传播优化空间的界定与分析

4.1 编码格式与解析方法

本节将通过图3所示的五层网络（A至E）实际案例，详细阐述编码方法的语义原理、其在硬件实现中的具体表现，以及不同编码方案所涉及的技术权衡。

如图3所示，该编码格式包含六个属性，可分为两大类：层融合相关属性（LFA）和DRAM读写相关属性（DLSA）。因此，整体解析过程也分为两个阶段。第一阶段通过解析 LFA 来确定：1）计算粒度与执行顺序，以及芯片上复用张量的缓冲区占用情况；2）所有需要与DRAM交互的存储单元。第二阶段则通过解析DLSA，推导出所有需加载/存储至DRAM的张量的具体时序参数及缓冲区占用细节。

4.1.1 层融合相关属性

LFA 包括***层序、细粒度层融合切割（FLC）、瓦片数量及DRAM切割（DC），***这些内容将逐一介绍。

第一个属性“层序”将这些层按序列顺序排列，表示其粗粒度执行顺序。因此，有效的层序不能存在从右向左的依赖关系，否则会导致早期计算层所需的数据尚未完成计算的情况。其格式为向量。图3展示了层序示例：交换D与E的顺序仍可形成有效层序，但交换A与B则无法形成有效层序。

第二个属性Fine-grained Layer-fusion Cut（FLC）记录切割位置，将图层划分为细粒度层融合组（FLG）。所有FLC共同构成 FLC 集合（FLCS）。每个 FLG 具有**Tile Number属性**，用于确定 FLG 的计算粒度。 FLG 内的图层以瓦片为单位进行连续处理，而非先完成第一层再开始第二层，从而节省片上缓冲区开销。如图3左上角示例所示，图层被切割为三个FLG（A、B、C、E、D），其Tile Number分别为1、2、2。在 FLG（C、D、E）中，每个图层被分割为两个瓦片并依次处理。我们采用启发式策略为各层分配瓦片编号的分割维度：优先进行批次分割，其次确定输出特征图的高度和宽度（尽量保持等宽以减少重叠）。未分割通道维度的原因在于，若分割通道维度将导致后续处理流程受阻。

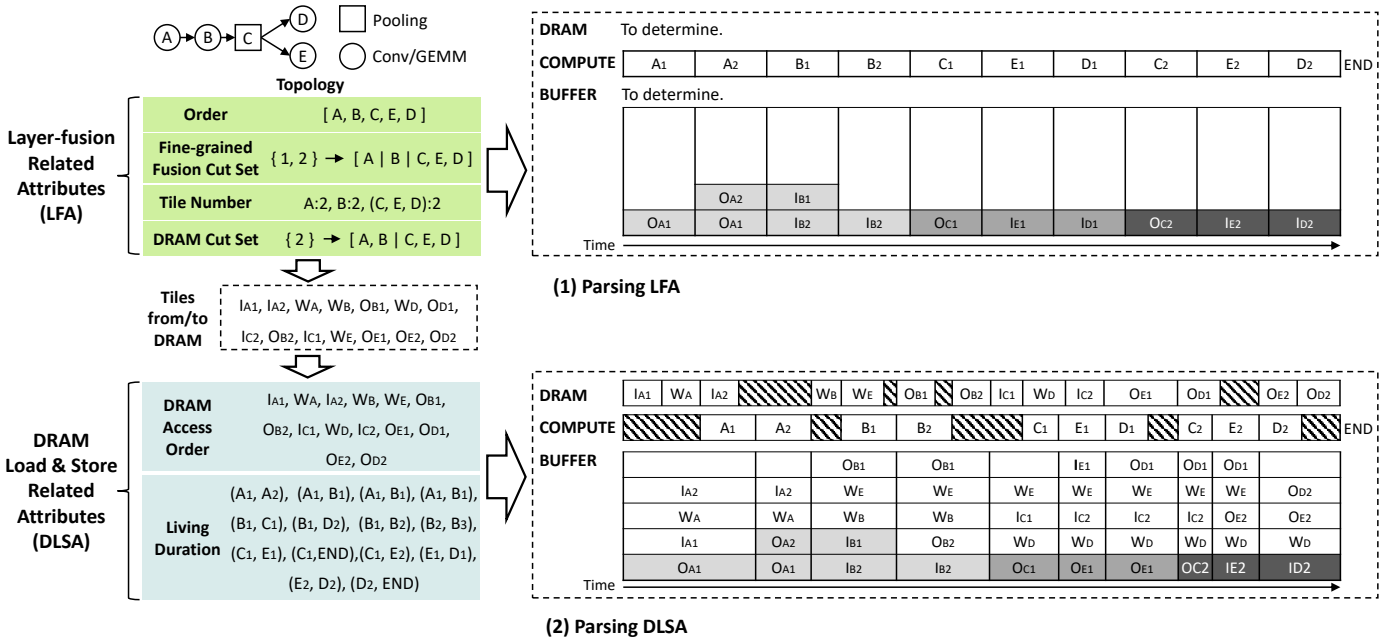


Figure 3: Parsing an Example Encode of five layers into Actual Scheduling Schemes. The DRAM, COMPUTE, and BUFFER in the right part show the DRAM access, workload computation, and buffer usage, respectively. We use M_i to represent the ith tile of layer M, and $I/W/O_{M_{(i)}}$ to represent the Ifmaps, Weights, and Ofmaps of $M_{(i)}$. In the BUFFER, blocks with the same background color represent the same data.

layer from accessing all channels, making it impossible to fuse multiple layers. For example, in Fig.2, with a batch size of 1 and a tile number of 4, we split the height and width dimensions each by 2 (the channel dimension is not shown). It is worth emphasizing that when the intermediate layers involve operations such as convolution or pooling, which produce overlaps, the size of each tile may be larger than $1/Tile$ of the size of the fmaps (e.g., Layer A and B in Fig. 2). The specific method for determining each tile size of intermediate layers has already been proposed in Cocco [32] and DeFENIS [26], so we directly adopt their strategy. The trade-off involved with the Tile Number primarily concerns the balance between buffer usage, overlap overhead, and computational efficiency within the core. Finer-grained tiles imply reduced buffer usage, but if they involve layers that produce overlaps, they may incur greater computational and memory access overheads. Additionally, the computational efficiency within the core may decrease (as smaller tiles imply a reduced scheduling space).

By parsing the above three attributes, we can derive the entire computation sequence (the COMPUTE row in Fig. 3).

DRAM Cut records the cut locations to divide the layers into Layer-fusion Groups (LGs), with dependencies between different LGs needing to be first sent to DRAM and then retrieved for computation. All DRAM Cuts constitute the DRAM Cut Set (DCS). For example, B has a dependency on C, and there is a DRAM cut between B and C. Therefore, the Ofmaps of B needs to be sent to DRAM, and C needs to retrieve the Ifmaps from DRAM. Thus, All the requirements for each tile's interaction with DRAM can be determined as follows: if a layer has weights, then its weight requirements are loaded from DRAM. For example, except for C which does not have weights, the weights of all other layers need to be loaded from DRAM (W_M in the left part of Fig. 3). If a layer has forward dependencies that span different LGs (or its input is the overall network input), then the ifmaps of its related tiles need to be loaded from DRAM (I_{M_I}) in the left part of Fig. 3). If it has backward dependencies that span different DCGs (or is the overall network output), then the ofmaps of its related tiles need to be written back to DRAM (O_{M_I} in the left part of Fig. 3). The trade-off involved in DCS primarily concerns the balance between buffer requirements and the volume of DRAM access. Generally speaking, the more fused layers there are (the fewer the DCS), the lower the number of DRAM accesses will be, but the demand for buffer capacity will increase.

The remaining data dependencies that do not cross a DRAM cut can be directly reused on-chip. The buffer occupancy duration for such data extends from the production of the Ofmaps tile to the consumption of the tile. For example, if A's Ofmaps are consumed by two tiles of B, its buffer occupancy is shown in Fig. 3(1). In Fig. 3(1), we only show the buffer allocation for on-chip data transfers, while all data related to communication with DRAM are not displayed (to be determined in the DRAM Load & Store Phase).

Since the DCS is a subset of the FLCS, this means that each DCG contains one or more FLGs. Within a DCG, each FLG is a fine-grained multi-layer fusion unit. If there are data dependencies between different FLGs, the producing FLG needs to aggregate the Ofmaps from different tiles before they can be used by the consuming FLG. Once the computation for an FLG is completed, its weights can be released and

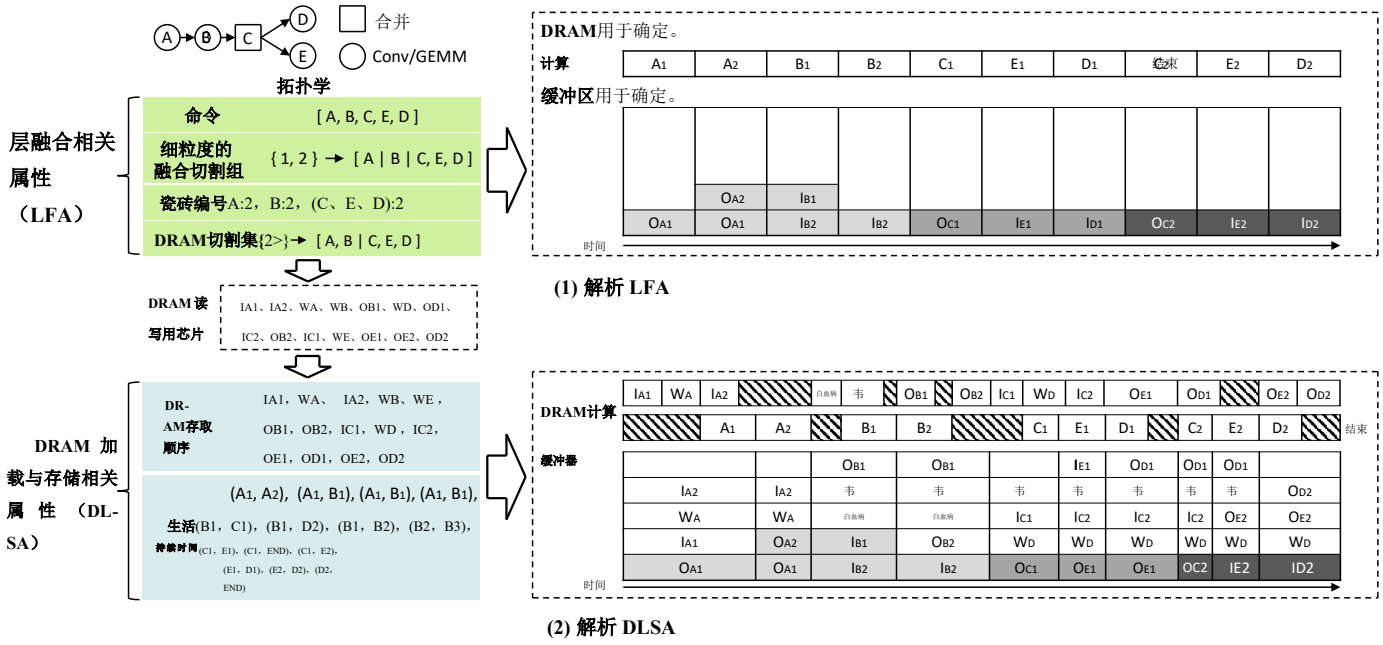


图3：将五层示例编码解析为实际调度方案。右侧的DRAM、计算单元和缓冲区分别显示DRAM访问、工作负载计算和缓冲区使用情况。我们用Mi表示层M的第i个瓦片，用$I/W/OM_{(i)}$表示M的Ifmaps、Weights和Ofmaps$_{(i)}$。在缓冲区中，背景颜色相同的区块代表相同数据。

无法访问所有通道的层，导致无法融合多层结构。例如，在图2中，当批量大小为1且瓦片数量为4时，我们将高度和宽度维度各进行2次分割（通道维度未显示）。需要强调的是，当中间层涉及卷积或池化等会产生重叠操作时，每个瓦片的尺寸可能超过功能映射尺寸的*Tile*分之一（如图2中的层A和B）。中间层各瓦片尺寸的具体确定方法已在Cocco[32]和DeFENIS[26]中提出，因此我们直接采用其策略。瓦片数量的权衡主要涉及缓冲区使用率、重叠开销与核心计算效率之间的平衡。更细粒度的瓦片意味着缓冲区使用率降低，但若涉及产生重叠的层，则可能带来更高的计算开销和内存访问开销。此外，核心计算效率可能下降（因更小的瓦片意味着调度空间减少）。

通过解析上述三个属性，我们可以推导出完整的计算序列（图3中的计算行）。DRAM切割记录了将层划分为层融合组（LGs）的切割位置，不同LG之间的依赖关系需先发送至DRAM，再从DRAM中获取以进行计算。所有DRAM切割共同构成DRAM切割集（DCS）。例如，B依赖于C，且B与C之间存在DRAM切割。因此，B的Ofmaps需发送至DRAM，而C需从DRAM中获取Ifmaps。由此可确定每个瓦片与DRAM交互的所有需求：若某层具有权重，则需满足其权重需求。

从DRAM中加载。例如，除了C层不包含权重外，其他所有层的权重都需要从DRAM中加载（图3左半部分的*WM*）。若某层的前向依赖跨越不同LG（或其输入为整个网络输入），则其相关瓦片的ifmaps需要从DRAM中加载（图3左半部分的I_{M_I}）。若其后向依赖跨越不同DCG（或为整个网络输出），则相关瓦片的ofmaps需要写回DRAM（图3左半部分的*OMI*）。DCS涉及的权衡主要在于缓冲区需求与DRAM访问量之间的平衡。一般来说，融合层越多（DCS越少），DRAM访问次数越少，但对缓冲容量的需求会增加。

那些未跨越DRAM切割线的剩余数据依赖关系可直接在芯片上复用。此类数据的缓冲区占用时长从Ofmaps瓦片生成开始，持续到该瓦片被消耗为止。例如，若A的Ofmaps被B中的两个瓦片使用，其缓冲区占用情况如图3(1)所示。图3(1)仅展示了芯片内部数据传输的缓冲区分配情况，而与DRAM通信相关的所有数据均未显示（具体将在DRAM加载与存储阶段确定）。

由于DCS是 FLCS 的子集，这意味着每个 DCG 包含一个或多个FLG。在 DCG 内部，每个 FLG 都是一个细粒度的多层融合单元。若不同FLG之间存在数据依赖关系，生成 FLG 需先聚合来自不同瓦片的Ofmap，才能供消费 FLG 使用。当 FLG 的计算完成后，其权重便可被释放。

no longer occupy the buffer. For example in Fig. 3, the weights of A can be released after its two tiles are computed. B needs to wait until all Ofmaps of A are computed before it can start its computation. Additionally, the Ofmaps of A (which serve as the Ifmaps for B) need to be stored on-chip and consumed progressively. Combining the above analysis, it can be seen that the FLG primarily involves a trade-off in buffer occupation. FLG can free up some of the buffer space occupied by weights, but it will cause the fmaps to accumulate and be consumed gradually.

4.1.2 DRAM-Load-and-Store-related Attributes

DLSA includes **Tensor Order and Living Duration**, which will be introduced one by one.

The **Tensor Order** attribute determines the access sequence for all tensors that need to communicate with DRAM, such as the tensors in the dashed squares of Fig. 3.

Every tensor that requires communication with DRAM has a adjustable **Living Duration** attribute, which is a 2-element tuple (start,end). Both start and end are tile IDs. This attribute has dual implications: First, within this time frame, buffer is allocated to this tensor. For example in Fig. 3, the Living Duration of W_E is (B_1, D_2). Therefore, it is allocated buffer space for storage within Tiles B_1 and $D2$. Second, it schedules the timing for loading and storing. Specifically, the start indicates when the tensor can begin to be loaded/stored, and the end marks when the tensor must complete transmission; otherwise, subsequent compute tasks cannot start. For example, the *end* of O_{E_1} is C_2, so if its store has not completed, C_2 cannot start. However, the actual start time is more complex; loading/storing data can begin at the start tile ID, but the specific loading time depends on whether the required data is ready and whether the loads and stores preceding this tensor in the Tensor Order have been completed. For example, the *Start* of I_{C_2} is C_1, but the loading of the preceding W_D takes a long time, causing the actual load of I_{C2} to begin only in the middle of E_1.

4.2 Space Size Comparison

From the preceding discussion, we can infer that in our notation, each scheme is encoded by six variable attributes. Thus, our notation constructs an approximately six-dimensional vast optimization space. In contrast, if we map the scheduling from the state-of-the-art (SOTA) Cocco [32] into our notation, only Layer Order and DRAM Cut are variable (with FLC identical with DRAM Cut), and the scheduling corresponding to the other four attributes is determined by heuristic strategies, making its explorable optimization space much smaller than ours. To bridge the gap and fully explore this vast optimization space, uncovering its potential for optimization, we introduce our Two-stage optimization framework, Tensor Shuttle, in the following chapter.

5. TENSOR SHUTTLE FRAMEWORK

5.1 Tensor Shuttle Overview

As shown in Fig. 4, Tensor Shuttle is an end-to-end DNN scheduling framework. Tensor Shuttle takes as inputs: (1) a DNN model description file produced by high-level frameworks like PyTorch, which is further processed by the Model

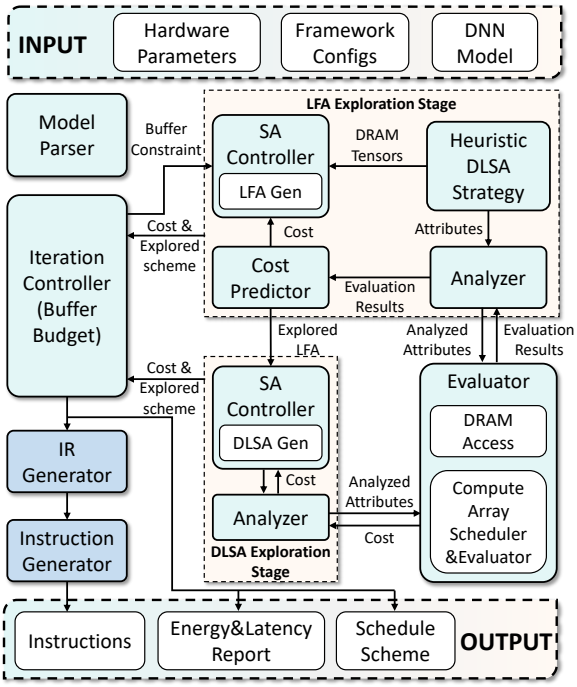


Figure 4: Tensor Shuttle Framework

Parser to extract the necessary information for scheduling; (2) hardware configuration (computing power, DRAM bandwidth, buffer size, and so on); (3) framework settings (optimization goal, searching hyperparameters, and so on). After scheduling, Tensor Shuttle outputs: (1) energy cost and performance reports; (2) a detailed scheduling scheme; (3) instructions.

The optimization objective of Tensor Shuttle is configurable as $Energy^n \times Delay^m$, where n and m are adjustable parameters that prioritize energy efficiency and performance according to different needs. Energy and latency refer to the total energy consumption and delay for processing a batch of samples. Therefore, to evaluate the effectiveness of Tensor Shuttle in latency-centric scenarios, small batch sizes can be used, while larger batch sizes can be employed for throughput-centric scenarios.

As shown in Fig. 4, the main search process employs a two-stage approach, separately varying and searching LFA and DLSA in each stage, with each stage using a simulated annealing algorithm for independent searches. The reason for using a two-stage search is that, as analyzed in Sec. 4.1, changes in LFA significantly impact DRAM communication requirements, making it difficult to retain good solutions obtained by varying DLSA attributes while continuously varying LFA. To better consider the mutual influence of the two stages, we equip the LFA Exploration Stage with a Cost Predictor. This predictor can correct the schemes of the first stage, which use basic prefetch and store strategies, by predicting the capabilities of the second stage and providing the corrected cost. This allows the first stage to take into account the second stage's capabilities, avoiding the cost of optimizing DRAM access requirements in the first stage that could be optimized in the second stage, while also providing better optimization

不再占用缓冲区。例如在图3中，A的权重值在计算出其两个瓦片后即可释放。*B*需要等待A的所有Ofmap计算完成才能开始运算。此外，A的Ofmap（作为*B*的Ifmap）需存储在芯片上并逐步消耗。综合上述分析可见，该 FLG 主要涉及缓冲区占用的权衡问题。虽然 FLG 能释放部分由权重值占用的缓冲区空间，但会导致fmap逐渐累积并被逐步消耗。

4.1.2 与DRAM加载与存储相关的属性

DLSA 包括**张量阶数与存活时长**，将逐一进行介绍。

张量顺序属性决定了所有需要与DRAM通信的张量的访问顺序，例如图3中虚线方框内的张量。

每个需要与DRAM通信的张量都具有可调节的**存活时长**属性，该属性是一个由两个元素组成的元组（起始时间，结束时间）。起始时间和结束时间均为瓦片ID。该属性具有双重含义：首先，在此时间范围内，缓冲区会被分配给该张量。例如在图3中，*WE*的存活时长为（*B1，D2*），因此其缓冲空间被分配存储在瓦片*B1*和*D2*中。其次，它会调度加载与存储的时间节点。具体而言，起始时间表示张量可开始加载/存储的时刻，结束时间则标记张量必须完成传输的截止点；否则后续计算任务将无法启动。例如，*OE 1*的*结束*时间是*C2*，若其存储操作未完成，则C_2无法启动。但实际起始时间更为复杂：数据加载/存储可在起始瓦片ID处开始，但具体加载时间取决于所需数据是否已就绪，以及张量顺序中位于该张量之前的加载与存储操作是否已完成。例如，IC2的起始of I_{C_2}，但前序*WD*的加载耗时较长，导致I_{C2}的实际负载仅在E_1的中间阶段开始。

4.2 空间大小比较

通过前文讨论可以推断，在我们的符号体系中，每个调度方案均由六个变量属性进行编码。因此，该符号体系构建了一个近似六维的庞大优化空间。相比之下，若将当前最先进的 SOTA 调度算法Cocco[32]映射到我们的符号体系中，仅能将层序和DRAM切割作为变量（其FLC 与DRAM切割相同），其余四个属性对应的调度方案需通过启发式策略确定，这使得其可探索的优化空间远小于我们的体系。为弥合这一差距并充分挖掘该庞大优化空间的优化潜力，我们将在下一章提出双阶段优化框架Tensor Shuttle。

5. 张量穿梭框架

5.1 Tensor Shuttle概述

如图4所示，TensorShuttle是一个端到端 DNN 调度框架。TensorShuttle的输入包括：(1)由PyTorch等高级框架生成的DNN 模型描述文件，该文件随后由Model进行进一步处理。

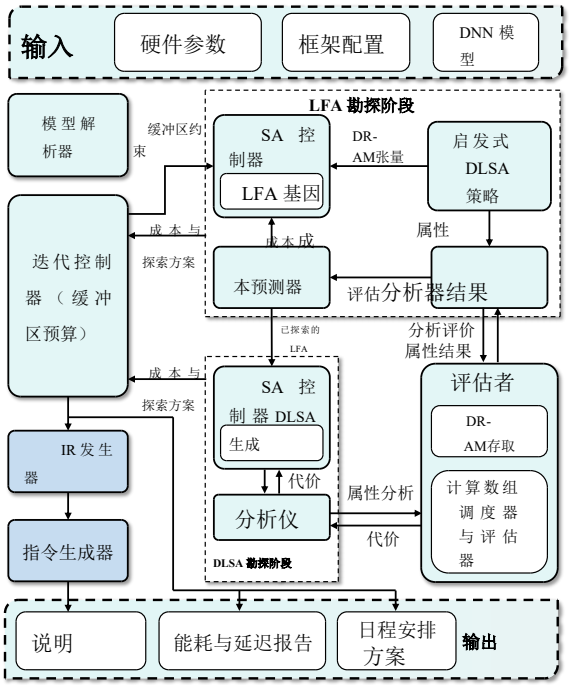


图4：Tensor Shuttle框架

解析器用于提取调度所需信息；(2) 硬件配置参数（包括计算能力、DRAM带宽、缓冲区容量等）；(3)框架参数设置（涵盖优化目标、超参数搜索策略等）。调度完成后，Tensor Shuttle将输出以下结果：(1)能耗成本与性能评估报告；(2)详细调度方案；(3)执行指令集。

Tensor Shuttle的优化目标可配置为*能量消耗×延迟时间*，其中n和m是可调参数，可根据不同需求优先考虑能效与性能指标。能量消耗和延迟时间分别指处理一批样本所需的总能耗与处理延迟。因此，在评估Tensor Shuttle在以延迟为中心的场景中的有效性时，可采用小批量处理方案；而在以吞吐量为中心的场景中，则可采用大批量处理方案。

如图4所示，主要搜索过程采用两阶段策略：每个阶段分别对 LFA 和 DLSA 进行参数调整与搜索，且各阶段均采用模拟退火算法进行独立搜索。选择两阶段搜索的原因在于，如第4.1节分析所示， LFA 参数的变化会显著影响DRAM通信需求，导致在持续调整 LFA 参数的同时难以保持通过 DLSA 属性调整获得的优质解。为更准确评估两阶段间的相互影响，我们在 LFA 探索阶段配置了成本预测器。该预测器可通过预判第二阶段性能并提供修正成本，对采用基础预取与存储策略的第一阶段方案进行优化调整。这种设计使第一阶段能够充分考虑第二阶段的性能特征，既避免了在第一阶段优化DRAM访问需求时产生的额外成本（这些需求本可在第二阶段进行优化），同时提升了整体优化效果。

potential for the second stage (for example, executing FLGs with ample DRAM bandwidth and buffer space before and after an FLG with severe DRAM bottleneck).

Outside this two-stage search, there is an Iteration Controller, which dynamically allocates buffer budgets to both competing stages based on the buffer usage status and optimization effects of the best solutions found in the previous iteration. This ensures that the optimization potential of both stages is fully realized, preventing one stage from having excessive buffer resources while the other is insufficient. Specifically, each iteration involves a complete exploration of the two-stage DRAM communication scheduling space.

The optimal results can be directly outputted from the Buffer Controller as scheduling schemes and reports, or they can be converted into a more easily parsable intermediate representation (IR) through our IR Generator module. This IR can then be input into the Instruction Generator module to produce actual instructions. We have already built and successfully run such a compilation flow for our in-house commercial accelerator.

5.2 Evaluator

The default Evaluator is based on the general hardware template introduced in Sec. 2. It can evaluate the generated scheduling schemes for different networks under various hardware parameters, assessing their energy cost and performance. As shown in Fig. 4, its input includes an ordered sequence of compute tile tasks, where each compute tile contains information about its layer, dimensions, dependencies, etc. Additionally, there is an ordered sequence of DRAM communication requests, where each request includes (start, end), dependencies, type (ifmaps, weights, or ofmaps), dimensions, etc.

The evaluation of overall energy consumption and performance follows a local-to-global approach. First, each compute tile and DRAM load/store request is evaluated individually, and then the overall assessment is conducted.

For each computing tile, the Core Array Scheduler & Evaluator performs scheduling search and power & performance evaluation, selecting the optimal scheduling scheme for the core array. The corresponding energy consumption and computing time of the searched optimal scheme are considered the tile's energy consumption and computing time. There has been extensive research on mapping a tile to a core array [12, 13, 19, 22, 25, 29, 34, 36, 37], so we use a classic maestro-like [22] search-based method for the scheduling exploration. The evaluator also adopts a maestro-like approach [22, 29]. Specifically, the energy consumption calculation involves counting the read/write operations of each level of the buffer, the number of various computations, and the data transmission volume on various interconnects, then multiplying each by the corresponding unit energy consumption and summing them up. Performance evaluation is conducted using an event-driven approach.

For each DRAM communication tensor, its energy consumption is calculated by multiplying the read and write data volumes by their respective unit energy consumption for DRAM reads and writes, and then summing them up. The read and write time is calculated as data volume divided by bandwidth.

The overall total energy consumption is calculated by summing up the energy consumption of the above subcomponents. The total computation time is derived based on the evaluated times of all computing tiles and DRAM communication tensors, using the following rule.

For each DRAM communication requirement, it can start execution only when the following three conditions are met: 1) The preceding requirement has been completed, 2) Its start time is greater than or equal to the current tile ID, 3) If the request is for ofmaps, it must wait until the generating computing tile has finished. For example, in Fig. 3, W_B has a start of B_1. Although the previous request has already been completed, it must still wait until A_2 finishes before it can begin. Similarly, although I_{C2}'s start is C_1, the preceding request is not completed until E_1, so it can only start at E_1.

For each computing tile task, it can start execution only when the following two conditions are met: 1) All required data (ifmaps, weights, parameters, etc.) are available. For example, A_1, B_1, and C_1 cannot follow the preceding computation tasks immediately because the required data are not available at the end of the preceding task. 2) The end of the preceding tile's DRAM communication request must be completed. For instance, C_2 cannot follow D_1 because O_{E1}'s end is D_1, and D_1 cannot start until OE1 has finished execution.

5.3 Exploration Stage

The exploration of the entire DRAM communication scheduling space is divided into two stages, with each stage using a standard simulated annealing algorithm. Specifically, in each iteration, the SA controller selects an operator to transform the encode with a certain probability and then analyzes and evaluates the transformed scheme. If the overall cost is lower, the change is accepted; otherwise, it is accepted with a probability that decreases as the number of iterations increases. Next, we will introduce the specific SA operators and strategies for the two stages.

5.4 LFA Exploration Stage

In the first exploration stage (LFA Exploration Stage), the SA operators primarily transform the LFA, while the DLSA is determined using a heuristic strategy (as introduced in Sec. 3.3). The specific operators are introduced below:

Change Layer Order: Randomly select a layer that has another layer with which it can exchange order, and swap their orders.

Change Tile Nmuber: Randomly select an FLG and either multiply or divide its Tile Number by 2.

Add/Delate An FLC: Randomly select a position with or without a cut to add or remove an FLC. Specifically, adding an FLC results in two FLGs, each inheriting the tile number attribute from the original FLG. Removing an FLC merges two FLGs into one, with the new FLG's tile number inherited probabilistically based on the layer count ratio of the original two FLGs.

Add/Delate An DC: Select a position within an FLCS that is or is not a DC, and add or remove a DC.

5.5 DLSA Exploration Stage

5.6 Validation

第二阶段的潜在方案（例如：在发生严重DRAM带宽瓶颈的 FLG 前后，使用具备充足DRAM带宽和缓冲空间的FLG执行操作）。

在这两阶段搜索机制之外，还存在一个迭代控制器。该控制器会根据缓冲区使用状态及前次迭代中找到的最佳解决方案的优化效果，动态分配缓冲区预算给两个竞争阶段。这种机制能确保两个阶段的优化潜力得到充分释放，避免某一阶段缓冲资源过剩而另一阶段资源不足的情况。具体而言，每次迭代都会对两阶段DRAM通信调度空间进行全面探索。

最优结果可直接从缓冲控制器输出为调度方案和报告，也可通过我们的中间表示生成器模块转换为更易解析的中间表示（IR）。该中间表示随后可输入指令生成器模块以生成实际指令。我们已为内部商用加速器成功构建并运行了此类编译流程。

5.2 评估者

默认评估器基于第2节介绍的通用硬件模板构建。该模型能够针对不同硬件参数下的各类网络生成调度方案进行评估，同时分析其能耗成本与性能表现。如图4所示，其输入数据包含按顺序排列的计算片任务序列（每个计算片均包含层级信息、维度参数、依赖关系等数据），以及按顺序排列的DRAM通信请求序列（每条请求包含起始/结束时间、依赖关系、数据类型（如ifmap、权重或ofmap）、维度参数等信息）。

整体能耗与性能评估采用从局部到全局的方法。首先对每个计算单元及DRAM加载/存储请求进行独立评估，随后开展整体性能分析。

对于每个计算片，核心阵列调度器与评估器会执行调度搜索及功耗与性能评估，为核心阵列选择最优调度方案。所搜索最优方案对应的能耗与计算时间即被视为该片的能耗与计算时间。已有大量研究探讨片映射至核心阵列的映射关系[12，13，19，22，25，29，34，36，37]，因此我们采用经典的maestro类[22]基于搜索的调度探索方法。评估器同样采用maestro类方法[22，29]。具体而言，能耗计算涉及对缓冲区各层级的读写操作次数、各类计算量以及各互连上的数据传输量进行统计，再分别乘以对应单位能耗并求和。性能评估采用事件驱动方法进行。

针对每个DRAM通信张量，其能耗计算方法为：将读写数据量分别乘以其对应的DRAM读写单位能耗值，再进行累加。读写时间则通过数据量除以带宽得出。

总能耗通过累加上述各子组件的能耗计算得出。总计算时间基于所有计算单元及DRAM通信张量的评估时间，采用以下规则进行推导。

针对每个DRAM通信需求，只有当满足以下三个条件时方可启动执行：

1) 前一项要求已满足，2）其开始时间大于或等于当前瓦片ID，3）若请求为ofmaps，则必须等待生成计算瓦片完成。例如，在图3中，*WB*的开始时间of B_1。尽管前一项请求已完成，仍需等待*A2*完成方可开始。同理，虽然$I_{C\,2}$的开始时间是*C1*，但前一项请求需待*E1*完成方可启动，因此只能在E_1开始。

对于每个计算瓦片任务，只有满足以下两个条件时才能开始执行：1) 所有所需数据（ifmaps、权重、参数等）均已可用。例如，*A1*、*B1*和C_1无法立即跟随前序计算任务执行，因为前序任务结束时所需数据尚未可用。2) 前序瓦片的DRAM通信请求必须完成。例如，C_2无法跟随D_1执行，因为*OE1*的结束时间是*D1*，且D_1必须待OE1执行完毕后才能启动。

5.3 勘探阶段

对DRAM通信调度空间的全面探索分为两个阶段，每个阶段均采用标准模拟退火算法。具体而言，在每次迭代中，SA控制器会以特定概率选择操作符对编码方案进行转换，随后对转换后的方案进行分析评估。若整体成本更低则接受该变更；若成本较高，则接受概率会随着迭代次数增加而递减。接下来我们将详细阐述这两个阶段中具体的模拟退火操作符及优化策略。

5.4 LFA 勘探阶段

在第一探索阶段（LFA 探索阶段），SA算子主要对 LFA 进行转换，而 DLSA 则通过启发式策略确定（如第3.3节所述）。具体算子如下：**改变层顺序**：随机选择一个可与其他层交换顺序的层，并交换它们的顺序。

更改瓦片编号：随机选取一个 FLG，将其瓦片编号乘以2或除以2。

添加/删除 FLC：随机选择带或不带切割的位置以添加或移除 FLC 。具体而言，添加 FLC 会产生两个FLG，每个FLG继承原始 FLG 的瓦片编号属性。移除 FLC 会将两个FLG合并为一个，新 FLG 的瓦片编号会根据原始两个FLG的层计数比例进行概率性继承。

添加/删除DC：在 FLCS 中选择一个属于或不属于DC的位置，然后添加或移除DC。

5.5 DLSA 勘探阶段

5.6 验证

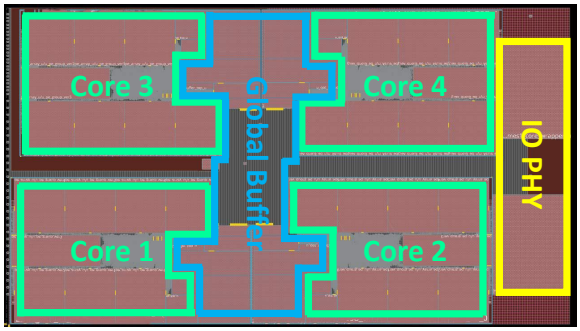


Figure 5: Layout of A Commercial Accelerator Using Tensor Shuttle

We validate the evaluator of Tensor Shuttle with the results from the Synopsys ZEBUServer4 Emulation System, which is the verification platform for our chip currently in tape-out (Fig. 5). Using ResNet50 [11] and Transformer [33], the average error is 5.65%.

5.7 Generality and Portability of Tensor Shuttle

The proposed encoding and Tensor Shuttle framework exhibit excellent versatility for two main reasons: 1. The template depicted in Figure 1 is highly general, encompassing many accelerators from both the industry and academia [16, 17, 35] add some citation; 2. The hardware behavior information encoded by the encoding is very general (i.e., the size of computation, loading, and storing, as well as instruction dependencies dispatch timing).

Our Tensor Shuttle also possesses excellent portability, which is due not only to the aforementioned reasons but also because our framework is designed with a robust modular architecture. Different accelerators may have distinct core micro-architectures, so it is only necessary to replace the corresponding Core Array Scheduler and Evaluator modules, as well as the instruction generation module within the Evaluator, to adapt to various accelerators. We have developed a comprehensive flow for our accelerator (Fig. 5), which serves as an exemplary guide for the process of porting to other accelerators.

6. EVALUATION

6.1 Experiments Setup

REFERENCES

[1] M. Alwani, H. Chen, M. Ferdman, and P. A. Milder, "Fused-layer CNN accelerators," in *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 2016, pp. 22:1–22:12. [Online]. Available: https://doi.org/10.1109/MICRO.2016.7783725

[2] P. Bannon, G. Venkataramanan, D. D. Sarma, and E. Talpes, "Computer and redundancy solution for the full self-driving computer," in *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 2019, pp. 1–22. [Online]. Available: https://doi.org/10.1109/HOTCHIPS.2019.8875645

[3] J. Cai, Y. Wei, Z. Wu, S. Peng, and K. Ma, "Inter-layer scheduling space definition and exploration for tiled accelerators," in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, Y. Solihin and M. A. Heinrich, Eds. ACM, 2023, pp. 13:1–13:17. [Online]. Available: https://doi.org/10.1145/3579371.3589048

[4] Y. Chen, J. S. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 2016, pp. 367–379. [Online]. Available: https://doi.org/10.1109/ISCA.2016.40

[5] Y. Chen, T. Yang, J. S. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, 2019. [Online]. Available: https://doi.org/10.1109/JETCAS.2019.2910232

[6] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, "Dadiannao: A machine-learning supercomputer," in *47th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2014, Cambridge, United Kingdom, December 13-17, 2014*. IEEE Computer Society, 2014, pp. 609–622. [Online]. Available: https://doi.org/10.1109/MICRO.2014.58

[7] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: https://doi.org/10.18653/v1/n19-1423

[8] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021.

[9] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, "Shidiannao: shifting vision processing closer to the sensor," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture, Portland, OR, USA, June 13-17, 2015*, D. T. Marr and D. H. Albonesi, Eds. ACM, 2015, pp. 92–104. [Online]. Available: https://doi.org/10.1145/2749469.2750389

[10] A. Firoozshahian, J. Coburn, R. Levenstein, R. Nattoji, A. Kamath, O. Wu, G. Grewal, H. Aepala, B. Jakka, B. Dreyer, A. Hutchin, U. Diril, K. Nair, E. K. Aredestani, M. Schatz, Y. Hao, R. Komuravelli, K. Ho, S. Abu Asal, J. Shajrawi, K. Quinn, N. Sreedhara, P. Kansal, W. Wei, D. Jayaraman, L. Cheng, P. Chopda, E. Wang, A. Bikumandla, A. Karthik Sengottuvel, K. Thottempudi, A. Narasimha, B. Dodds, C. Gao, J. Zhang, M. Al-Sanabani, A. Zehtabioskuie, J. Fix, H. Yu, R. Li, K. Gondkar, J. Montgomery, M. Tsai, S. Dwarakapuram, S. Desai, N. Avidan, P. Ramani, K. Narayanan, A. Mathews, S. Gopal, M. Naumov, V. Rao, K. Noru, H. Reddy, P. Venkatapuram, and A. Bjorlin, "Mtia: First generation silicon targeting meta's recommendation systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3579371.3589348

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778. [Online]. Available: https://doi.org/10.1109/CVPR.2016.90

[12] K. Hegde, P. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, "Mind mappings: enabling efficient algorithm-accelerator mapping space search," in *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021*, T. Sherwood, E. D. Berger, and C. Kozyrakis, Eds. ACM, 2021, pp. 943–958. [Online]. Available: https://doi.org/10.1145/3445814.3446762

[13] Q. Huang, A. Kalaiah, M. Kang, J. Demmel, G. Dinh, J. Wawrzynek, T. Norell, and Y. S. Shao, "Cosa: Scheduling by constrained optimization for spatial accelerators," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 554–566. [Online]. Available: https://doi.org/10.1109/ISCA52012.2021.00050

[14] M. James, M. Tom, P. Groeneveld, and V. Kibardin, "ISPD 2020 physical mapping of neural networks on a wafer-scale deep learning accelerator," in *ISPD 2020: International Symposium on Physical*

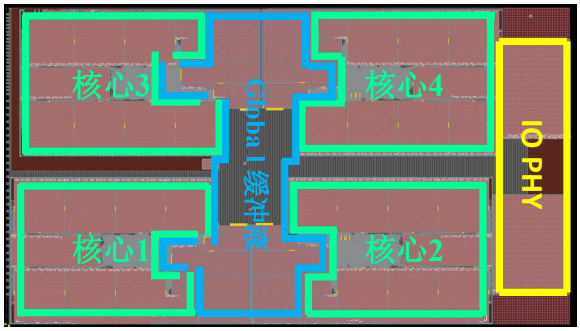


图5：商业加速器布局示意图
张量穿梭机

我们通过Synopsys ZEBUServer4仿真系统（该系统是当前处于tape-out阶段的芯片验证平台，见图5）的结果验证Tensor Shuttle评估器。采用ResNet50[11]和Transformer[33]模型时，平均误差为5.65%。

5.7 Tensor Shuttle的通用性与可移植性研究

所提出的编码与Tensor Shuttle框架展现出卓越的通用性，主要基于两个原因：1. 图1所示模板具有高度通用性，涵盖了来自工业界和学术界的多种加速器[16，17，35]添加引用；2. 编码所编码的硬件行为信息具有高度通用性（即计算规模、加载与存储规模，以及指令依赖性调度时序）。

我们的Tensor Shuttle不仅具备出色的可移植性，这不仅源于前文所述原因，更得益于其框架采用的稳健模块化架构设计。由于不同加速器可能具有不同的核心微架构，因此只需替换对应的核心阵列调度器、评估器模块以及评估器内部的指令生成模块，即可适配各类加速器。我们为加速器开发了完整的流程体系（图5），该体系可作为向其他加速器移植过程的示例指南。

6. 评估

6.1 实验设置

参考文献

[1] M. Alwani、H. Chen、M. Ferdman 和 P. A. Milder，《融合层CNN加速器》，收录于*第49届IEEE/ACM国际微架构研讨会（micro 2016），2016年10月15-19日，台湾台北*。IEEE计算机学会，2016年，第22卷：1-12页。[在线]。可获取地址：https://doi.org/10.1109/micro.2016.7783725

[2] P. Bannon、G. Venkataramanan、D. D. Sarma与E. Talpes合著的《全自动驾驶计算机的计算机与冗余解决方案》，收录于*2019年IEEE热芯片第31届研讨会（HCS），美国加利福尼亚州库比蒂诺市，2019年8月18-20日*。IEEE，2019年，第1-22页。[在线]。可获取于：https://doi.org/10.1109/hotchips.2019.8875645

[3] J. Cai、Y. Wei、Z. Wu、S. Peng 和 K. Ma，“平铺加速器中的层间调度空间定义与探索”，载于*会议论文集第50届国际计算机体系结构年会，ISCA 2023年6月17日至21日在美国佛罗里达州奥兰多市举行*。Y. Solihin与M. A. Heinrich编著，ACM出版社，2023年，第13卷：1-17页。[在线版]。可获取地址：https://doi.org/10.1145/3579371.3589048

[4] Y. Chen、J. S. Emer与V. Sze合著的《Eyeriss：卷积神经网络高效数据流的空间架构》，收录于*第43届ACM/IEEE国际计算机体系结构年会（ISCA 2016年6月18-22日，韩国首尔）*。IEEE计算机学会，2016年，第367–379页。[在线]。可获取于：https://doi.org/10.1109/ISCA.2016.40

[5] Y. Chen、T. Yang、J. S. Emer与V. Sze合著的《Eyeriss v2：移动设备上新兴深度神经网络的灵活加速器》，*IEEE新兴与选择性电路系统期刊*，第9卷第2期，第292–308页，2019年。[在线]。可获取于：https://doi.org/10.1109/JETCAS.2019.2910232

[6] Y. Chen、T. Luo、S. Liu、S. Zhang、L. He、J. Wang、L. Li、T. Chen、Z. Xu、N. Sun与O. Temam合著的《Dadiannao：一台机器学习超级计算机》，收录于*第47届IEEE/ACM国际微架构研讨会（micro 2014），英国剑桥，2014年12月13-17日*。IEEE计算机学会，2014年，第609–622页。[在线]。可获取地址：https://doi.org/10.1109/micro.2014.58

[7] J. Devlin、M. Chang、K. Lee 和 K. Toutanova，《BERT：用于语言理解的深度双向变换器预训练》，载于*《2019年计算语言学协会North American 分会会议论文集：人类语言技术，NAACL- HLT 2019，美国明尼苏达州明尼阿波利斯市，2019年6月2日至7日，第1卷（长篇与短篇论文）》*，J. Burstein、C. Doran与T. Solorio，编. 计算语言学协会，2019年，第4171–4186页. [在线]. 可获取于:https://doi.org/10.18653/v1/n19-1423

[8] A. 多索维茨基、L. 贝尔、A. 科列斯尼科夫、D. 魏森伯恩、X. 赵、T. 乌特海纳、M. 德赫加尼、M. 明德勒、G. 海戈尔德、S. 格利、J. 乌什科雷特及 N. 霍尔斯比，《一张图片胜过16×16个单词：大规模图像识别中的Transformer模型》，2021年。

[9] Z. 杜、R. Fasthuber、T. 陈、P. Ienne、L. 李、T. 罗、X. 冯、Y. 陈和O. Temam，《视点消隐：将视觉处理更贴近传感器》，收录于*第42届国际计算机体系结构年会论文集，美国俄勒冈州波特兰市，2015年6月13-17日*，D. T. Marr与D. H. Albonesi主编，ACM出版社，2015年，第92–104页。[在线]。可获取地址：https://doi.org/10.1145/2749469.2750389

[10] A. 菲罗兹沙希安、J. 科伯恩、R. 利文斯坦、R. 纳托吉、A. 卡马斯、O. 吴、G. 格鲁瓦尔、H. 阿帕拉、B. 雅卡、B. 德雷尔、A. 哈钦、U. 迪里尔、K. 纳尔、E. K. 阿雷斯塔尼、M. 沙茨、Y. 郝、R. 科穆拉维利、K. 霍、S. 阿布·阿萨尔、J. 沙杰拉维、K. 奎因、N. 斯里达哈、P. 坎萨尔、W. 韦、D. 杰亚拉曼、L. 常、P. 乔普达、E. 王、A. 比库曼德拉、A. 卡尔提克·森戈图维尔、K. 托滕普迪、A. 纳拉辛哈、B. 多兹、C. 高、J. 张、M. 阿尔-萨纳巴尼、A. 泽塔比奥斯库伊、J. 费克斯、H. 余、R. 李、K. 冈德卡尔、J. 蒙哥马利、M. 曹、S. 德瓦拉卡普拉姆、S. 德赛、N. 阿维丹、P. 拉马尼、K. 纳拉亚南、A. 马修斯、S. 戈帕尔、M. 纳乌莫夫、V. 拉奥、K. 诺鲁、H. 雷迪、P. 文卡塔普拉姆，以及 A. Bjorlin，《Mtia：第一代硅靶向元推荐系统》，收录于*第50届国际计算机体系结构年会论文集*，ISCA ‘23系列。美国纽约州纽约市：计算机协会，2023年[在线版]。可获取地址：https://doi.org/10.1145/3579371.3589348

[11] K. He，X. Zhang，S. Ren与J. Sun，《深度残差学习在图像识别中的应用》，收录于*2016年IEEE计算机视觉与Pattern Recognition 会议（CVPR 2016），美国内华达州 Las Vegas 市，2016年6月27-30日*。IEEE计算机学会，2016年，第770–778页。[在线]。可获取地址：https://doi.org/10.1109/CVPR.2016.90

[12] K. Hegde、P. Tsai、S. Huang、V. Chandra、A. Parashar 及 C. W. Fletcher，“心智映射：实现高效的算法加速器映射空间搜索，” 收录于ASPLOS *'21: 第26届ACM国际编程语言与操作系统架构支持会议，虚拟活动，美国，2021年4月19-23日，*T. Sherwood，E. D. Berger与C. Kozyrakis编，ACM出版社，2021年，第xx页。943–958。[在线]。可获取于：https://doi.org/10.1145/3445814.3446762

[13] Q. 黄、A. 卡拉亚、M. 康、J. 德梅尔、G. 辛、J. 瓦夫日涅克、T. 诺雷尔和Y. S. 肖，《Cosa：基于约束优化的空间加速器调度》，收录于*第48届ACM/IEEE国际计算机体系结构年会，ISCA 2021，西班牙瓦伦西亚，2021年6月14-18日*。IEEE，2021年，第554–566页。[在线]。可获取地址：https://doi.org/10.1109/ISCA52012.2021.00050

[14] M. James、M. Tom、P. Groeneveld和V. Kibardin，“ISPD 2020晶圆级深度学习加速器上的神经网络物理映射”，收录于*ISPD 2020：国际物理映射研讨会*